

Trading Engine — To-Do List

Updated 15 September 2026. Replaces the previous list.

How to read this. Every job from the old list is still here. Jobs you have finished are in **Done** at the end. **NEW** means the job is not on the old list. **UPDATED** means the job changed because of what we found this week.

No deadlines are set. The old list had none, and you have not given me any. See [Questions for me](#).

Effort tags like **1-2 DAYS** come from the project guide. They are engineering estimates only, they overlap, and they exclude building the engine itself. They are not learning deadlines.

Do these first

Five jobs. The first three are quick. The last two unblock everything else.

- 1 Protect your capture files.** Your recorded market data was wiped from the laptop this week and had to be pulled back from iCloud. It will happen again.
- 2 Save this week's decisions into ADR 0014.** You answered five design questions in chat. They are not written down anywhere. If you lose the chat, you lose them.
- 3 Commit the work sitting on your laptop.** Code fixes and document updates are not saved to git yet.
- 4 Answer the last three design questions.** ADR 0014 cannot be accepted until these are done. Nothing else in the engine can start.
- 5 Make the loader check the capture file is complete.** Right now a damaged or empty capture loads silently and replay runs on it. This is the biggest correctness risk.

A. Protect what you have

Small jobs. Do them first because they stop you losing work.

☐ **Protect your capture files from being wiped**

HIGH

NEW

NO DEADLINE

Your project sits in `~/Desktop`, which syncs to iCloud. iCloud removes the contents of big files it thinks you are not using, and leaves an empty shell behind. That is what happened to your capture data. The files looked normal but read as empty.

Your capture files are not in git (they are too big), so git cannot bring them back.

STEPS

1. Choose one: move the whole project out of `~/Desktop`, or move just the `data/` folder somewhere iCloud does not sync.
2. Copy the `data/` folder to a second place — an external drive or a backup service that is not iCloud.
3. Turn off "Optimise Mac Storage" in System Settings → Apple Account → iCloud, if you keep the project where it is.
4. Check nothing is still hollow.

COMMANDS YOU WILL NEED

```
# list files iCloud has emptied out
find data -type f -flags +dataless

# pull one back
brctl download data/raw/<capture>/segment-0000.jsonl
```

13 files under `data/` were still hollow after the last check.
 Biggest at risk: `data/raw0ld/btcusd-live-orders.jsonl` (126 MB).
 "dataless" is the macOS flag meaning "contents are not on this laptop".

Done when: `find data -type f -flags +dataless` returns nothing, and a full copy of `data/` exists somewhere outside iCloud.

☐ **Commit the work sitting on your laptop**

HIGH

NEW

NO DEADLINE

There are code fixes, document updates and two new PDFs that are not saved to git.

WHAT IS WAITING

- Six small code and comment fixes (listed in **Done**).
- Updated `README.md`, `TOD0.md`, `status.md` and `docs/project-progress-guide.md`.
- `docs/TradingEngine-DeepDive.pdf` — the 54-page code manual.
- This list, once you are happy with it.

Suggested split: one commit for the review-driven document updates, one for the code fixes, one for the PDFs.

Done when: `git status` is clean, and the test suite still passes 304 of 304.

☐ **Delete the broken build folder**

LOW

NEW

NO DEADLINE

The `build/` folder in your project no longer works. It fails before it starts with `FindThreads` only works if either `C` or `CXX` language is enabled. Your source code is fine — only the saved build settings are broken.

```
rm -rf build
cmake -S . -B build -DCMAKE_BUILD_TYPE=RelWithDebInfo
cmake --build build -j
```

Done when: `cmake --build build -j` works from a clean start and `ctest --test-dir build` passes 304 of 304.

B. Finish the design decision

This is the gate. No engine code can be written until ADR 0014 is accepted. An "ADR" is a short document recording one design decision and why you made it.

☐

Write one page of behaviour before naming anything in code

HIGH

NEW

NO DEADLINE

This is your next working session. Write down what should happen in each situation below, **before** you pick any type or enum names. Small scenarios decide how the classes behave. Writing more class skeletons will not settle these questions.

Situation	What you must decide
Trusted, open market, trading allowed	When is watching finished and a decision asked for?
Trusted, open market, but trading is scripted off	Are new orders blocked? What happens to orders already out?
Trusted market with only one side quoted	Does watching carry on? What blocks a decision?
A gap, then re-syncing	What is held, what is thrown away, and what makes it ready again?
An order arrival and a cancel at the same moment	Which happens first, and can a fill still happen?

Keep two things apart: the book being *broken inside your own code*, and a *gap in the data*. The first should stop the run outright — you cannot fix broken code state by fetching another snapshot. Block decisions whenever the market looks crossed, and do not invent a how-long-before-we-panic number without evidence.

Then write one complete event timeline — from a market message arriving to the account changing — and put the money example in section D into your own words.

Done when: one page describes the outcome for all five situations, plus one complete event timeline from market message to account change.

☐

Save this week's five answers into ADR 0014

HIGH

NEW

NO DEADLINE

You settled five of the eight open questions in conversation. The ADR still lists all eight as open. Write the answers in before they are lost.

THE FIVE ANSWERS TO WRITE DOWN

- Operational state:** do not build it yet. At this stage there is no operator, no kill switch, and a file cannot disconnect — so it could only ever hold one value and no test could prove it works. The decision gate checks two things for now: is the data trustworthy, and is the market a shape you can trade.
- Latency queue:** an order only joins the queue after both the decision gate and the risk check say yes. The queue models travel time to the exchange, not thinking time. A rejected order never enters it.
- First fill:** an order can fill the moment it arrives, using the book as it stands at that exact moment.
- Time order:** use the exchange's own timestamp only — never the time your machine received it, because that changes with your network speed. If a real market event and your own order share a timestamp, the real event goes first. If two of your own orders share one, the one submitted first goes first.
- Money order:** confirm the fill, work out the fee, then — when buying, add the price and fee into a new average cost; when selling, use the old average cost to work out profit, and take the fee off that profit. Update cash, position, average cost and profit together, so nothing ever sees half an update. Selling part of a holding does not change the average cost of what is left.

File: docs/decisions/0014-event-loop-causality-and-decision-authority.md
Status today: **proposed**. It becomes **accepted** once all eight are answered.

Done when: the ADR lists these five as decided, not open, and the "still to decide" list has only three items left.

☐ **Answer the last three design questions**

HIGH

UPDATED

NO DEADLINE

This was item 1 on the old list. Five of eight questions are now answered, so only three are left.

WHAT IS STILL OPEN

1. **Does the decision gate still get asked when the data is untrusted?** Either always ask it, so every block has a written reason, or skip it to save time. Your own evidence rule says every blocked decision must be counted by name, which points to always asking.
2. **Which rejection reasons become permanent promises?** Either only the ones a test checks by name, or everything that exists when you ship.
3. **How long must the market look wrong before you stop trusting the book?** A "crossed" book means the best buy price is above the best sell price, which should be impossible. Your own note says do not invent a number without evidence. The way forward is to block every time it happens, record how long each one lasts, and pick a number once you have data.

Done when: one accepted ADR answers all eight questions, says what is deliberately left out, includes at least one hand-worked timeline, and gives a named reason for every blocked decision and rejected order.

C. Repair the foundation

This was item 2a on the old list. This week gave it real evidence and exact locations. Do this before building more on top.

☐ **Make the loader check the capture file is complete**

HIGH

UPDATED

NO DEADLINE

Your recorder already writes down exactly what it produced. The C++ loader reads none of it. So a damaged, empty or half-written capture loads without complaint, and replay runs on it.

This week proved it: the capture file was empty, the loader returned "success, zero events", and the real problem only appeared much later as a confusing test failure.

WHAT THE RECORDER ALREADY RECORDS, AND NOBODY CHECKS

```
payload_bytes , payload_sha256 — size and fingerprint of the data file
frames_bytes , frames_sha256 — same for the index file
frames , order_events , trade_events , control_frames — expected counts
chain_valid — whether the recorder saw any gaps
status — whether the recording finished properly
Example: the 22 August capture declares 14,684,968 bytes and 29,490 frames.
```

STEPS

1. Decide where the check lives. Three options, in the Questions section.
2. Read the size and count fields in `manifest_reader.cpp`. It currently reads only file paths.
3. Compare them against the real files before handing any events to replay.
4. Give every mismatch its own named error, so the message says what was wrong.
5. Make an empty capture a named failure instead of a silent success.
6. Add a test with a deliberately truncated capture.

```
Files: src/capture/manifest_reader.cpp , src/capture/segment_loader.cpp
The silent-success line is segment_loader.cpp:101.
```

Done when: a capture with a wrong size, wrong count or empty payload is rejected with a named reason, and replay cannot run on it.

☐ **Fix the test that should skip but fails instead**

MEDIUM

NEW

NO DEADLINE

One test is meant to skip quietly when your private capture data is missing. It only checks that `manifest.json` exists. That small file survived; the big data file it points to did not. So the test ran anyway and reported a hard failure.

Check the data file is present and not empty, not just the manifest.

File: `tests/unit/test_bitstamp_joined_capture.cpp`, lines 349–354

Done when: deleting or emptying the capture data makes the test skip with a clear message, not fail.

☐ **Guard the two unchecked sums**

MEDIUM

UPDATED

NO DEADLINE

Two places add numbers together without first checking the result will fit. Everywhere else in your code checks first. In C++, a whole number going past its limit is undefined behaviour — the compiler is allowed to assume it cannot happen, so the result is not simply a wrong number.

Real market sizes are far too small to trigger this. The point is consistency with your own standard.

`src/feed/trade_reconciler.cpp:26` — adding up fills at the same timestamp
`src/capture/capture_coordinator.cpp:81` and `:84` — adding up checkpoint sizes
 Copy the pattern already in `src/book/price_level.cpp:11`.

Done when: both places check before adding, return a named error instead of overflowing, and a test proves it.

☐ **Finish the order book's undo path**

MEDIUM

UPDATED

NO DEADLINE

When adding an order, the book first creates a new price level, then adds the order to it. If the second step runs out of memory and throws, the cleanup code never runs, because that cleanup only sits on the normal return path. An empty price level is left behind, and the error escapes as an exception instead of the error value the function promises.

An empty price level is a problem on its own: it can become the "best price" with nothing actually for sale behind it.

Same gap exists when an order changes price.

`src/book/order_book.cpp:57` (add) and `:125` (price change)
 The throwing line is `src/book/price_level.cpp:15`.

DECIDE FIRST

Is running out of memory something to recover from, or something to stop on? Today the code promises a recoverable error but does not always deliver one. Pick one and make it consistent.

Done when: a failed add or price change leaves the book exactly as it was, with no leftover empty level, and a test forces the failure.

☐ **Run the Python tests in CI and pin the downloads**

MEDIUM

UPDATED

NO DEADLINE

CI runs only one Python test. Your recorder and capture-checker tests never run, so they can break without anyone noticing. Your C++ downloads also have no fingerprint, so you cannot prove you got the same file twice.

STEPS

1. Add the Python test folder to CI.
2. Add a `URL_HASH` to each download in `cmake/dependencies.cmake`. Your own ADR 0001 already asks for this.
3. Write down which Python version and which `websockets` version you need.
4. Add a Release-mode test run, because some checks switch off outside debug builds.

```
.github/workflows/ci.yml:28 — currently the only Python line
cmake/dependencies.cmake:13 and :19 — the two downloads
```

Done when: CI runs every Python test, both downloads have a fingerprint, and someone new can set up the project from the repository alone.

☐ **Make the release-mode safety check real, or say it is not**

LOW

NEW

NO DEADLINE

`validateStructure()` checks the order book is internally consistent. It is built entirely from assertions, which switch off in release builds. So in release the function walks the whole book and checks nothing.

Either make the checks work in release too, or write plainly that this is a debug-only check.

```
src/book/order_book.cpp:196 ; called from src/feed/bitstamp/replay.cpp:172
```

Done when: the release build either really checks, or the documents stop implying it does.

D. Build the trading engine

Unchanged from the old list except where marked. Do these in order — each one uses the one before. Do not start until ADR 0014 is accepted and section C is done.

☐ **Build the money tracker, tests first**

HIGH

NO DEADLINE

Track cash, position, average cost, profit taken, profit on paper, and fees. Use whole numbers only — never decimals, which drift.

Write a buy, a partial sell, a final sell and fees out by hand first. Then build the class to match your arithmetic.

WORKED EXAMPLE TO START FROM

From the project guide. Dollars shown; the code uses whole money units. On sale rows the execution price is the valuation mark.

Action	Cash	Units held	Total fees	Equity
Start	1,000	0	0	1,000
Buy 2 at 100, fee 1	799	2	1	999
Mark price moves to 105	799	2	1	1,009
Sell 1 at 110, fee 1	908	1	2	1,018
Sell final 1 at 90, fee 1	997	0	3	997

First sale makes 10 profit, last loses 10, so gross profit ends at zero. Fees explain the final loss of 3. At the 105 mark the open position shows 10 paper profit, which is not cash yet.

$$\text{equity} - \text{starting equity} = \text{realized gross PnL} + \text{unrealized PnL} - \text{fees}$$

This assumes no money added or taken out. Fees are already out of cash — do not subtract them from equity twice. These are teaching assumptions, not an agreed accounting policy yet.

DECIDE BEFORE YOU WRITE CODE

- Name the currency and scale for cash and fees. Price ticks are not money.
- Choose how average cost is stored. Buying 1 at 100 and 2 at 101 gives $302 \div 3$.
- Set rounding for part-units, order value and fees. Check the arithmetic mid-calculation.
- Define shorts, and what happens when a position crosses through zero.
- Give every execution an identity so the same fill cannot be counted twice.
- Refuse unsafe arithmetic without half-changing the account.

Done when: tests reproduce every hand calculation exactly, tell taken profit apart from paper profit, cover long, flat and short, and refuse numbers that are too big without half-changing anything.

☐ **Name the order types and rejection reasons**

MEDIUM

NO DEADLINE

Add only the small value types the accepted ADR needs: the order request, the operational state, the reason a decision was blocked, and the reason an order was rejected.

Keep three ideas separate: seeing the market, being allowed to decide, and being allowed to trade. Do not add an interface until something really needs it.

Done when: tests show that untrusted data, each untradeable market shape, and each scripted block produce their own distinct reason.

☐ **Add the venue seam and a pretend exchange**

MEDIUM

NO DEADLINE

Start with submit, accept, reject, partial fill, full fill and cancel. A cancel that has been asked for is not a cancel that has happened — test an order filling while its cancel is still in flight.

The risk check looks at size, value and the position you would end up with. It must count orders already accepted but not yet filled, and check the worst case in both directions without assuming opposite orders fill at the same time.

The pretend exchange checks safety again before accepting, and records why. Build the no-delay version first. Add the delay queue only as the ADR says.

Rate limits, loss limits, the production kill switch and live recovery stay out. Do not add empty stand-ins that always say yes.

WORKED EXAMPLE: A CANCEL THAT ARRIVES TOO LATE

Event, for an order of 10 units	Filled so far	Still tradeable
Accepted	0	10
Fill 4	4	6
Cancel requested	4	6
Another fill of 2 before the cancel lands	6	4
Cancel confirmed for the rest	6	0

Cancelling does not undo the 6 units already filled.

WORKED EXAMPLE: ORDERS THAT HAVE NOT FILLED YET

Your limit is 10 units. You hold 0. You already have an accepted buy for 6 that has not filled. Another buy for 6 would take you to 12 — over the limit. Checking only your current holding plus the new order misses this.

Set the exposure aside when the order is admitted, and release it on a fill, a confirmed cancel or a rejection. Test the worst case in both directions separately — do not assume a buy and a sell cancel each other out.

Done when: submit, accept, reject, fill and cancel can each be tested on their own, a rejection changes nothing, and a strategy cannot skip the risk check.

☐ **Add the strategy seam and a do-nothing strategy**

MEDIUM

NO DEADLINE

Keep watching the market separate from deciding to trade. A strategy gets a read-only view of the book that it must not keep, and returns order requests. It never gets the real book or the ability to trade directly.

Done when: the do-nothing strategy sees every event, asks for nothing, builds against the real interface, and can be tested without the engine.

☐ **Build the engine loop**

MEDIUM

NO DEADLINE

Wire together the feed, clock, strategy, decision gate, risk check, venue, order book, health tracker and money tracker. One thread. Follow the order the ADR sets before making anything faster.

Done when: a made-up timeline proves the book updates before the strategy sees it, watching continues while trading is blocked, decisions are gated, timing order is repeatable, and the money maths after a fill is exact. Test scenarios can be reused and can point at the first event where two runs differ.

☐ **Pass the Stage 5 proof tests**

MEDIUM

NO DEADLINE

Section C must already pass. Then prove the whole path:

- The do-nothing strategy changes no counts, cash, position or profit over a real capture.
- One scripted strategy places a real order, and your hand-worked fill, fee and profit match end to end.
- At least one real risk rule rejects an order and changes nothing. A rule that always says yes does not count.
- Ten identical runs produce identical fingerprints for events, book, decisions, orders, cash, position and profit.
- Every blocked decision and rejected order is counted by name.
- Partial fills, cancel races, outstanding exposure and duplicate fills all have saved test cases. Replaying the fill and fee list rebuilds the account.
- Mid-run comparisons catch differences that identical totals would hide.

WHAT EACH TEST IS ACTUALLY ASKING

Test	The question it answers
Do-nothing strategy over saved input	Did it see everything, and leave money untouched?
Scripted order plus hand calculation	Do intention, risk check, fill, fee and accounting agree?
Oversized order with an outstanding order	Does a real limit reject without changing anything?
Partial fill then cancel race	Do later fills and the cancelled remainder stay consistent?
Duplicate execution	Can the same fill post twice? It must not.
Ten identical runs	Do the same inputs give the same outputs?
Mid-run order-level comparison	Can equal totals hide different underlying orders?
Damaged capture and memory failure	Does it stop safely with a precise reason?

Three different claims, often confused: **repeatable** (same in, same out), **internally correct** (follows your own rules), and **realistic** (matches a real exchange). A made-up scenario proves the first two. Only an independent exchange snapshot speaks to the third, and neither proves your order would actually have filled.

Done when: every one of these passes from a fresh copy of the project, without needing your private capture files.

☐ **Ship the replay program and refresh the documents**

LOW

NO DEADLINE

Connect `apps/replay_main.cpp` to the build. It runs the saved capture through the real engine with the do-nothing strategy and prints a short report: where the data came from, counts, health, market shapes, decisions, orders, money and fingerprints.

Include a run record: input fingerprints, which commit and build, whether the project had uncommitted changes, instrument scales, settings, what was left out, policy versions and result fingerprints. Keep detailed step-by-step tracing optional.

Then update plan v4, the README, the handoff notes, and the placeholder comments so nothing still calls a finished module unfinished.

Done when: the build produces the replay program, its report comes out the same every time, and no current document calls a built module a placeholder.

E. Five upgrades worth building

From the project guide. These make your Stage 5 evidence much stronger. They assume the engine already exists, so they come after section D — but decide now whether you want them, because two of them change how you write the tests.

☐ **Build a reusable scenario runner**

MEDIUM

NEW

1-2 DAYS

NO DEADLINE

Write market events, control changes and expected outcomes as plain test data. Use the same scenarios in tests and later in the replay program. When two runs differ, name the first event where they disagree.

Start with the GoogleTest setup and saved examples you already have.

Done when: one scenario file can be run by both a test and the replay program, and a deliberate mismatch reports the first differing event.

☐ **Keep a fill and fee journal**

MEDIUM

NEW

1-2 DAYS

NO DEADLINE

Record execution ID, order ID, quantity, price, fee and the account change for every fill. Replay that list into a fresh account and check you get the same balances.

Decide what happens if the same execution shows up twice. Keep it in memory with a small export — you do not need a database.

Done when: replaying the journal rebuilds the account exactly, and a repeated execution cannot post twice.

☐ **Save a run record with every result**

MEDIUM

NEW

1 DAY

NO DEADLINE

Save input fingerprints, which commit and build produced the result, whether the project had uncommitted changes, instrument scales, settings, any random seed, what was left out, policy versions and output fingerprints.

Plan v4 section 8 already asks for this. A small JSON file is enough.

Done when: every result comes with a record that lets you rebuild the exact run that produced it.

☐ **Add a trace that explains the first difference**

MEDIUM

NEW

1-2 DAYS

NO DEADLINE

For each step record the input position, the logical time, the reason, and a short before and after summary. Compare orders and account state part-way through, not just the final totals.

Make the detailed trace something you switch on, so it does not slow down later speed measurements.

Done when: a failing comparison points at one event and explains what changed there.

☐ **Add awkward scenarios that stay repeatable**

LOW

NEW

1-2 DAYS TO START

NO DEADLINE

Keep direct test cases for gaps, duplicates, backward timestamps, same-time events, partial fills, cancel races and limits.

Then add generated sequences from a fixed random seed, checked against a simple independent model of what should happen. Borrow the idea before reaching for a new tool.

Done when: a generated failing case can be replayed exactly from its seed.

What can wait

Written down so you can stop worrying about it. Each of these needs Stage 5 finished first.

Later stage	Why it has to come after
Queue position and execution baselines	Needs a settled order lifecycle and written-down assumptions.
Held-out testing of trading ideas	Needs trustworthy labels and protection against peeking at future data.
Speed work and v3 tape replay	Needs a correct reference first, and matching results to compare against.
Live and paper trading operations	Adds outside actions, recovery and advanced controls.
Dashboard and showcase	Needs stable reports and an engine worth showing.

Your next milestone is **a trading simulation you can explain**. Profitable signals, several exchanges and lock-free queues are all outside it.

Questions for me

I did not guess at any of these. Answer them and I will fold them in.

1. Your message had an empty placeholder: "[paste new tasks, deadlines or changes]".

Nothing was pasted, so I added nothing from it. Do you have tasks or changes to add?

2. Do you want any deadlines?

There are no dates anywhere in the project, and you have not given me any, so every task says "No deadline". Tell me the dates and I will add them.

3. Where should the capture completeness check live?

Three options. (a) Inside `LoadSegment` — simplest, but then loading and checking are the same job. (b) A separate check that produces its own type, which replay is the only thing able to accept — this makes it impossible to replay unchecked data, but is the most work. (c) A step in the coordinator — cheap, but people can still call the loader directly and skip it. I would pick (b), because the mistake you just hit was exactly "something used data nobody checked".

4. Should the capture check get its own ADR?

It is a contract decision, like ADR 0013 was. It could be ADR 0015, or it could just live inside the section C task. I lean towards its own ADR, because three separate programs have to agree on the same rules.

5. Move the whole project out of iCloud, or just the `data/` folder?

Moving everything is simplest and safest. Moving only `data/` keeps your code backed up by iCloud, which is useful, but leaves the split to remember.

6. Is running out of memory recoverable, or fatal?

Needed before the order book undo path can be finished properly. Right now the code half-promises recovery. Either answer is fine; it just has to be one of them.

7. Should the 54-page code manual be committed to the repository?

It is at `docs/TradingEngine-DeepDive.pdf` and not yet in git. It is 2.3 MB.

Useful references

From the project guide. There is no single correct trading-engine design — these show how others solved the same problems. None of them decides anything for you.

S1 — Timing. NautilusTrader: Data. Keeps event time and arrival time apart, and warns about clock drift. Use it to think about what the strategy could know, not to replace your Bitstamp ordering.

S2 — Simulated fills. QuantConnect: Trade fills. Fill price and size, partial fills and stale prices. Shows why you must state your fill rules, not that your simulator matches a real exchange.

S3 — Money types. NautilusTrader: Value types. Fixed-point price, quantity and money. Your own scales and rounding are still your choice.

S4 — Order states. FIX: Order state changes. Where the pending-cancel and fill-during-cancel behaviour comes from. Borrowing the idea does not make the project FIX compliant.

S5 — Reserved balances. NautilusTrader: Accounting. Splits committed money from available money. Your outstanding-position limit still needs its own tests.

S6 — Run records. W3C PROV-DM. A formal model for tracking where results came from. Take the principle; you do not need the full standard.

- S7 — Model-based testing.** Hypothesis: Stateful tests. Compare a sequence of actions against a simpler model. Your existing GoogleTest scenarios can do this before adding a tool.
- S8 — Repeatable simulation.** NautilusTrader: Deterministic simulation testing. Seed-controlled, replayable failures.
- S9 — Python setup.** PyPA: Repeatable installs. Declare and pin your Python dependencies.
- S10 — Build downloads.** CMake: URL hashes. Verify downloaded archives.

Done

Finished work, newest first. Kept short.

This week

- ✓ **Recovered the capture data.** It was not lost — iCloud had emptied the files. One command brought it back. 15 Sep
- ✓ **Confirmed the test suite passes 304 of 304** from a clean build. 15 Sep
- ✓ **Removed dead code** in the order book — a duplicate line that could never run. 15 Sep
- ✓ **Corrected the fingerprint comments.** The code says "FNV-1a" but uses a slightly different starting number. The comment now says so, and warns not to change it — every saved fingerprint depends on it. 15 Sep
- ✓ **Corrected the "order-independent" label** on the applied-event fingerprint. It is order-sensitive, and that is deliberate. 15 Sep
- ✓ **Fixed the swapped labels in ADR 0008.** "Cancels ahead of you" is the optimistic case, not the pessimistic one. 15 Sep
- ✓ **Tidied two mismatched function names** in the replay code. 15 Sep
- ✓ **Recorded the fresh test result and the broken build folder** in the handoff notes. 15 Sep
- ✓ **Wrote the 54-page code manual** tracing one market message from raw bytes to finished book state. 15 Sep
- ✓ **Answered five of the eight open design questions** — operational state, latency queue, first fill, time ordering and money ordering. 14–15 Sep
- ✓ **Fixed the health tracker** so restarting from a corrupted state clears the old failure reason. Commit d00e244
- ✓ **Regenerated the old to-do PDF** to match the eight-item list. Commit ad0f0a7

Earlier foundation work

- ✓ **Merged the two decoders** into one pass, keeping the old order-only decoder for the legacy recorder.
- ✓ **Turned on the CI guards** for clock use and decimal numbers, with tests that deliberately break to prove they work.
- ✓ **Split the order book's internal checks** from its market-shape reporting.
- ✓ **Separated "is the data trustworthy" from "what shape is the market"**, with both written down in `CONTEXT.md`.
- ✓ **Made the order book handle each event type properly**, leaving state unchanged when an ordinary event is rejected, and checking itself after every successful change in debug builds.
- ✓ **Passed 303 tests** after the trust and shape rework. 9 Sep

Working agreement: `CLAUDE.md`. Active checklist: `TOD0.md`. Plain-language guide: `docs/project-progress-guide.md` (worked examples, effort estimates and the references above came from it). Code manual: `docs/TradingEngine-DeepDive.pdf`. Refresh this list when a task closes or when reality disagrees with it.